

Kaosar and Segments DSA

1. What is the problem asking?

You are given a **permutation** of $1 \dots n$.

For every contiguous subarray, we look at its maximum.

We need to count subarrays whose maximum is **not equal to the maximum of the whole array**.

Since the array is a permutation of $1 \dots n$, the biggest element of the entire array is always:

n

So the problem becomes:

Count the subarrays whose maximum is NOT n .

2. Key Observation ★

Because n appears **exactly once** in the permutation, suppose its position is:

$p = [\dots | n | \dots]$
 $\quad \quad \quad \uparrow$

Any subarray that **contains n** will have:

$\max(\text{subarray}) = n$

Therefore, such a subarray **doesn't count**.

So which subarrays do count?

They must **not contain n** .

That means the valid subarrays must be completely:

- on the **left** of n , or
- on the **right** of n .

For example:

$[1 \ 3 \ 5 \ 7 \ | \ 9 \ | \ 2 \ 4 \ 6 \ 8]$
 $\quad \quad \quad \uparrow$
 $\quad \quad \quad n$

The valid subarrays are:

$[1]$
 $[3]$
 $[5]$
 $[7]$

$[1, 3]$
 $[3, 5]$
 $[5, 7]$
 $\dots$

and similarly on the right.

3. How many subarrays are there in an array of length k ?

This is a very important formula:

Number of subarrays = $k * (k + 1) / 2$

Why?

For an array:

$[a \ b \ c]$

Subarrays starting at each position:

$a \rightarrow 3 \text{ choices}$

```
b    → 2 choices
c    → 1 choice
```

So:

$$3 + 2 + 1 = 6$$

which is:

$$3 * 4 / 2 = 6$$

4. Build the intuition step-by-step

Brute force idea

We could generate every subarray:

```
C++
for (int l = 0; l < n; l++) {
    int mx = 0;

    for (int r = l; r < n; r++) {
        mx = max(mx, p[r]);

        if (mx != n)
            ans++;
    }
}
```

This takes:

$$O(n^2)$$

which is too slow for $n = 100000$.

Can we avoid checking every subarray?

Yes.

We know the only element that can make the subarray's maximum equal to the whole-array maximum is:

n

And because it's a permutation, there is **only one** n .

So we don't care about the other values at all.

Just find where n is.

Suppose:

```
n = 9
p = [1 3 5 7 9 2 4 6 8]
      ↑
```

There are:

```
4 elements on the left
4 elements on the right
```

A valid subarray cannot cross the **9**.

Therefore:

```
left valid subarrays = 4 * 5 / 2 = 10
right valid subarrays = 4 * 5 / 2 = 10
```

Answer:

$$10 + 10 = 20$$

That's the entire trick.

5. Formula

Let's use **0-based indexing**.

Suppose n is at index pos .

Then:

```
left elements = pos
right elements = n - pos - 1
```

Therefore:

```
answer =
    left * (left + 1) / 2
    + right * (right + 1) / 2
```

Since the answer can be large, take modulo **9973**.

6. C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 9973;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int n;
        cin >> n;

        int pos = -1;

        for (int i = 0; i < n; i++) {
            int x;
            cin >> x;

            if (x == n) {
                pos = i;
            }
        }

        long long left = pos;
        long long right = n - pos - 1;

        long long ans = 0;

        ans += left * (left + 1) / 2;
        ans += right * (right + 1) / 2;

        ans %= MOD;

        cout << ans << '\n';
    }

    return 0;
}
```

7. Code Explanation

Find the position of n

```
C++
if (x == n) {
    pos = i;
}
```

Since the array is a permutation, n occurs exactly once.

Number of elements on the left

```
C++
long long left = pos;
```

Because we're using 0-based indexing.

Example:

```
index: 0 1 2 3 4
array: 1 3 5 7 9
        ↑
      pos=4
```

There are 4 elements before index 4.

Number of elements on the right

```
C++
long long right = n - pos - 1;
```

For:

```
[1 3 5 7 9 2 4 6 8]
          ↑
        pos=4
```

```
right = 9 - 4 - 1
       = 4
```

Count left subarrays

```
C++
left * (left + 1) / 2
```

If left = 4:

```
4 * 5 / 2 = 10
```

Count right subarrays

```
C++
right * (right + 1) / 2
```

Again:

```
4 * 5 / 2 = 10
```

Final answer

```
C++
ans = 10 + 10 = 20
```

8. Complete Dry Run

Input:

```
3
1
1
3
1 3 2
9
1 3 5 7 9 2 4 6 8
```

Test Case 1

```
n = 1
p = [1]
```

The maximum of the whole array is:

```
1
```

Position of 1:

```
pos = 0
```

Left:

```
left = 0
```

Right:

```
right = 1 - 0 - 1 = 0
```

Answer:

```
0 * 1 / 2 + 0 * 1 / 2
= 0
```

Output:

```
0
```

This makes sense because the only subarray is:

```
[1]
```

and its maximum is 1, which is the whole-array maximum.

Test Case 2

```
n = 3
p = [1 3 2]
```

n = 3 is at:

```
pos = 1
```

Array:

```
[1 | 3 | 2]
  ↑
  n
```

Left:

```
left = 1
```

Right:

```
right = 3 - 1 - 1
       = 1
```

Left subarrays:

```
1 * 2 / 2 = 1
```

That's:

```
[1]
```

Right subarrays:

```
1 * 2 / 2 = 1
```

That's:

```
[2]
```

Total:

```
1 + 1 = 2
```

Output:

```
2
```

Exactly as the sample says.

Test Case 3

```
n = 9
p = [1 3 5 7 9 2 4 6 8]
```

Find **9**:

```
index: 0 1 2 3 4 5 6 7 8
value: 1 3 5 7 9 2 4 6 8
          ↑
        pos=4
```

So:

```
left = 4
right = 9 - 4 - 1 = 4
```

Left side:

```
[1 3 5 7]
```

Number of subarrays:

```
4 * 5 / 2
= 10
```

Right side:

```
[2 4 6 8]
```

Number of subarrays:

```
4 * 5 / 2
= 10
```

Therefore:

```
answer = 10 + 10
        = 20
```

Output:

```
20
```

9. Why this solution is so simple

The important realization is:

The whole-array maximum is always n , and n occurs only once.

Therefore:

```
subarray contains n
      ↓
maximum = n
      ↓
INVALID
```

and:

```
subarray doesn't contain n
      ↓
maximum < n
      ↓
VALID
```

So we simply **split the array around n** .

```
      n
      ↓
[ LEFT | n | RIGHT ]
      ↓       ↓
count   count
subarrays subarrays
```

This converts an apparently $O(n^2)$ subarray problem into an $O(n)$ problem.

10. Complexity

For each test case, we scan the permutation once to find n .

Time

$O(n)$

Since:

 $\sum n \leq 100000$

overall complexity is:

 $O(\sum n) = O(100000)$

Space

We don't actually need to store the permutation.

 $O(1)$

extra space.

★ Pattern to remember

Whenever you see a problem like:

"Count subarrays whose maximum is different from the maximum of the entire permutation"

and the array is a permutation of $1 \dots n$, immediately think:

```
Global maximum = n
      ↓
Find position of n
      ↓
Valid subarray cannot cross n
      ↓
Count subarrays on left + right
```

Formula:

```
C++
long long left = pos;
long long right = n - pos - 1;

ans = left * (left + 1) / 2
      + right * (right + 1) / 2;
```

That is the core observation that turns this problem from brute force into a very easy $O(n)$ solution.

ChatGPT can make mistakes. Check important info.